



FIREBRAND

LYCEUM EDUCATION GROUP

Functions: Composing and Reusing Logic

Today is not about new syntax. It is about writing functions well enough to trust them.

What You Will Be Able To Do By This Afternoon

Six concrete skills you will practise and apply today.

- 1 Spot duplicated logic and recognise it as a signal, not just an annoyance.
- 2 Break one large task into two or more small, named functions.
- 3 Write a function that calls another function to get part of its job done.
- 4 Write a function that takes an array as a parameter and processes it.
- 5 Write a function that returns an object built from its inputs.
- 6 Judge whether a function is doing one job clearly, or several jobs vaguely.

One Rule, Six Places

The same eligibility rule, copied into six different files, changes on the seventh.

1

CONTEXT

A checker discovers that "18 or over and identity verified" is written out, slightly differently each time, in six separate scripts across a small codebase: a signup form, an account upgrade tool, a loan pre-check, and three others.

2

CHALLENGE

A new rule arrives: the age threshold is changing from 18 to 21. Someone has to update all six copies, by hand, without missing one.

3

TASK

Read the scenario and jot down, in your own words, what could go wrong with this update.

What Duplication Actually Costs

A function is not just shorter code. It is one place to fix, one place to trust.

1

One rule, one home: change the logic once, every caller gets the fix.

2

One name to remember: `isEligible` tells you what it checks without reading it.

3

One place to test: proving the function works once covers every call site.

4

One place a bug can hide: if something is wrong, there is exactly one place to look.

A Quick Reminder From Day 5

The shape of a function, exactly as introduced on Day 5.

```
eligibility-recap.js

1 function isEligible(age, verified) {
2   if (age >= 18 && verified) {
3     return true;
4   }
5   return false;
6 }
7 let result = isEligible(22, true);
8 console.log(result);
```

A Function Calling Another Function

One function using another function's result, instead of repeating its logic.

```
functions-calling-functions.js

1 function formatGrade(score) {
2   if (score >= 70) {
3     return "Distinction";
4   }
5   return "Pass";
6 }
7 function describeResult(name, score) {
8   let grade = formatGrade(score);
9   ... 2 more lines in the starter file
```

Activity 1 of 10: Trace the Call

Work out what two chained function calls actually produce, before you run anything.

- 1 Open trace-the-call.txt, showing two small functions and one call that chains them.
- 2 Trace the call by hand, writing down what each function returns.
- 3 Compare your answer with the people at your table before the group check.



TIMEBOX

6 minutes



OUTPUT

A written, step-by-step trace showing what each function returns and why.

Breaking a Big Job Into Small Jobs

The second half of today's skill: recognising when one function is secretly doing three jobs.

- 1 A function with "and" in its job description is a warning sign: validate the input and format the output and log the result.
- 2 Each small function should be nameable in a few words, without the word "and".
- 3 Splitting a job does not mean splitting it randomly: split where the sub-jobs are genuinely separate ideas.
- 4 A well-split function is easier to test, easier to reuse elsewhere, and easier for someone else to trust without reading every line.

One Condition, Now a Named Function

The same processApplication function before and after one condition becomes a named function

process-application-v1.txt

```
1 function processApplication(age, income) {
2   if (age < 18) {
3     return "Rejected: too young";
4   }
5   return "Approved";
6 }
```

process-application-v2.txt

```
1 function processApplication(age, income) {
2   if (!meetsAgeRule(age)) {
3     return "Rejected: too young";
4   }
5   return "Approved";
6 }
```

Activity 2 of 10: Name the Three Jobs

Read a description of one large function's job and decide how you would split it.

- 1 Open name-the-three-jobs.txt and read the function description.
- 2 Decide how many separate functions this should become, and name each one.
- 3 For each named function, write one sentence describing exactly what it does.



TIMEBOX

8 minutes



OUTPUT

A short list of proposed function names, each with a one-sentence job description.

Why Small, Single-Job Functions Are Easier to Trust

Hard to trust

- One function, 40 lines, three unrelated jobs tangled together
- A bug in the discount logic risks breaking the receipt message too
- Testing it means testing everything at once, every time

Easier to trust

- Three functions, each under ten lines, one clear job each
- A bug in the discount logic stays inside the discount function
- Testing each function alone is quick, and confirms exactly what broke

Activity 3 of 10: The Morning Build: From One Function to Three

Take one overloaded function and rebuild it as three functions that work together.

- 1 Open refactor-starter.js, containing one large function called reviewSubmission.
- 2 Identify the separate jobs reviewSubmission is doing.
- 3 Write two or three smaller, named functions, each doing exactly one of those jobs.
- 4 Rewrite reviewSubmission so it calls your new functions instead of repeating their logic.



TIMEBOX

16 minutes



OUTPUT

A rewritten file with two or three new named functions and a shortened reviewSubmission that calls them.

Check Your Own Work



Each of my new functions can be described in one sentence

No "and" needed in the description



My original function now calls my new functions

It does not repeat their logic anywhere



Have I actually run this and checked the output matches

Check it against at least one example from before the rewrite



Would a teammate understand each function's name without reading its body

If not, rename it before moving on

Morning in One Line



A function should do one job, and a bigger job is usually several functions working together.



Lunch

Back at the top of the hour. Bring your rewritten refactor-starter.js with you.



FIREBRAND

LYCEUM EDUCATION GROUP

Welcome Back

This afternoon: functions that work with the arrays and objects you already know.

A Function That Takes an Array

A function whose parameter is an entire array, not a single value.

```
total-scores.js

1 function totalScores(scores) {
2   let total = 0;
3   for (let i = 0; i < scores.length; i++) {
4     total = total + scores[i];
5   }
6   return total;
7 }
8 let classScores = [62, 78, 91, 54];
9 console.log(totalScores(classScores));
```

Activity 4 of 10: Predict What Comes Back

Trace a function that loops over an array, before checking your answer.

- 1 Open predict-array-function.txt, showing a function and one call to it.
- 2 Work out what the function returns, writing down your reasoning step by step.
- 3 Do not run any code, just reason through it on paper first.



TIMEBOX

6 minutes



OUTPUT

A written prediction with the reasoning that led to it.

A Function That Returns an Object

A function whose job is to build and hand back a whole object.

```
build-learner-record.js

1 function buildLearnerRecord(name, score) {
2   let record = {
3     name: name,
4     score: score,
5     passed: score >= 50
6   };
7   return record;
8 }
9 ... 2 more lines in the starter file
```

Activity 5 of 10: Write a Function That Builds a Record

Write a function that takes a few values in and returns a complete object.

- 1 Open build-record-brief.txt, describing what the new function needs to do.
- 2 Write a function called buildOrderSummary that takes the described inputs.
- 3 Have it return an object with the properties named in the brief.



TIMEBOX

10 minutes



OUTPUT

A working buildOrderSummary function returning a correctly shaped object.

A Function That Loops Over an Array of Objects

Bringing functions, loops, arrays and objects together in one place.

```
count-passed.js

1 function countPassed(learners) {
2   let count = 0;
3   for (let i = 0; i < learners.length; i++) {
4     if (learners[i].passed) {
5       count = count + 1;
6     }
7   }
8   return count;
9   ... 2 more lines in the starter file
```

Activity 6 of 10: Complete the Missing Function

Finish a function whose signature and intended use are already given.

- 1 Open complete-the-function.txt and read the empty function.
- 2 Read the comment describing what the function should do.
- 3 Write the function body so it returns the correct result.



TIMEBOX

10 minutes



OUTPUT

A completed function that correctly processes the given array of objects.

Checklist for a Good Function

Four honest questions worth asking about any function before calling it finished.

1

Can I describe its job in one sentence, without the word "and"?

2

Does its name actually say what it does, without needing to read the body?

3

Does it return a value rather than just printing one, wherever the caller might need that value again?

4

Would I trust it if a colleague handed it to me with no other explanation?

Activity 7 of 10: The Afternoon Build: A Mini Reporting Toolkit

Write a small set of functions that work together to summarise a cohort's results.

- 1 Open toolkit-brief.txt and read what each function should do.
- 2 Write filterPassed(learners), returning only the learner objects where passed is true.
- 3 Write averageScore(learners), returning the array's average score.
- 4 Write buildReport(learners), calling both functions above.



TIMEBOX

18 minutes



OUTPUT

Three working functions, plus a buildReport function that combines their results into one object.

Activity 8 of 10: Check It Against the Checklist

Swap your toolkit with another table and check it against today's four questions.

- 1 Swap your completed toolkit.js with another table or pair.
- 2 Check their functions against today's four checklist questions.
- 3 Give one specific piece of praise and one specific suggestion.



TIMEBOX

8 minutes



OUTPUT

Written feedback on another group's toolkit, referencing the four checklist questions directly.

Activity 9 of 10: What Got Easier Today?

- Which part of today felt most different from how you wrote functions before today?
- Was it splitting one big function into several, or writing a function around an array or object?
- What is one function you have already written, on an earlier day or outside this programme, that you would now rewrite differently?

Discuss briefly with the people at your table before the close.

What You Can Now Do

A quick stocktake of what today added to the functions skill first met on Day 5.

- 1 Recognise duplicated logic as a signal that something should become a function.
- 2 Split one overloaded function into several smaller, clearly named ones.
- 3 Write a function that calls another function to complete part of its job.
- 4 Write functions that take arrays as input and functions that return whole objects.
- 5 Judge whether a function is doing one job clearly enough to trust.

Activity 10 of 10: Four Part Exit Ticket

Answer four short questions about today before you leave.

- 1 Name one sign that tells you a function is doing more than one job.
- 2 Describe, in one sentence, an array-input versus an object-return function.
- 3 Name one function from today's toolkit you are proud of, and say why.
- 4 Name one habit from today you will try to keep using after this programme ends.



TIMEBOX

8 minutes



OUTPUT

**Four short written answers
reflecting on today's function-
design skills.**

Closing

You can now write functions you would trust someone else to use.



Duplicated logic is a signal to write a function, not a habit to accept.



A big job is usually several small, clearly named functions working together.



Functions can take arrays as input and return whole objects as output.



A function you can describe in one sentence is a function you can trust.